Phase 10 Structured Notes: Queue Using Two Stacks

1. Current Progress Before Phase 10

Before Phase 10, you already learned many queue ideas and implementations.

A queue follows this main rule:

```
FIFO = First In, First Out
```

This means:

```
The first value inserted into the queue is the first value removed from the
queue.
```

You also learned that a normal queue has two main ends:

Queue Part	Simple Meaning
`front`	The side where deletion happens
`rear`	The side where insertion happens

So in a normal queue:

```
Insert at rear.
Delete from front.
```

You have already studied these phases:

Phase	Main Topic	Simple Meaning
Phase 1	Queue fundamentals	Learned FIFO, front, rear, enqueue, and dequeue
Phase 2	Array queues	Compared single-pointer and two-pointer queues
Phase 3	Linear array queue in C	Built a complete C queue using `struct`, `malloc`, and functions
Phase 4	Queue in C++ OOP	Built the same queue using a class and object methods

Phase	Main Topic	Simple Meaning
Phase 5	Drawbacks of linear queues	Learned that linear queues waste memory
Phase 6	Circular queue	Solved memory wastage using circular movement and modulo
Phase 7	Linked-list queue	Built a queue using connected nodes
Phase 8	Double ended queue / deque	Learned a queue that can work from both ends
Phase 9	Priority queue	Learned a queue where priority decides deletion order
Phase 10	Queue using two stacks	Learn how two stacks can work together to behave like one queue

Phase 10 is the final phase because it proves an important idea:

```
A queue is defined by its behavior, not by the internal storage method.
```

In simple words:

```
As long as the structure behaves like FIFO, it can be called a queue.
```

A queue can be built using:

- an array
- a circular array
- a linked list
- a priority system
- a deque
- two stacks

Phase 10 shows that even though a stack normally behaves differently from a queue, two stacks can work together to create queue behavior.

---

2. Main Goal of Phase 10

The goal of Phase 10 is to learn how to build a queue using two stacks.

A stack follows this rule:

```
LIFO = Last In, First Out
```

A queue follows this rule:

```
FIFO = First In, First Out
```

At first, this looks strange.

A stack removes the newest value first. A queue removes the oldest value first.

So the main question is:

```
How can two LIFO stacks create one FIFO queue?
```

The answer is:

```
Use one stack for insertion and another stack for deletion.
```

---

3. Queue Rule Reminder

A queue removes values in the same order they were inserted.

Example:

```
Insert: 6, 3, 9, 5
Remove: 6, 3, 9, 5
```

The value `6` entered first, so `6` must leave first.

That is FIFO.

---

4. Stack Rule Reminder

A stack removes the most recent value first.

Example:

```
Push: 6, 3, 9, 5
Pop: 5, 9, 3, 6
```

The value `5` was pushed last, so `5` leaves first.

That is LIFO.

---

5. Queue vs Stack

Structure	Rule	Meaning	Example
Queue	FIFO	First inserted value leaves first	People waiting in a line
Stack	LIFO	Last inserted value leaves first	Plates stacked on top of each other

A queue and a stack are opposites in how they remove elements.

That is why Phase 10 is interesting:

```
We use two opposite-behaving structures to create queue behavior.
```

---

6. Big Idea of Queue Using Two Stacks

To create a queue using two stacks, we use:

```
S1 and S2
```

Each stack has a different job.

Stack	Job
`S1`	Used for enqueue. New values are pushed into `S1`.
`S2`	Used for dequeue. Old values are popped from `S2`.

Simple mental model:

```
S1 = waiting room for new items
S2 = exit room for old items
```

The queue uses `S1` to collect new elements. The queue uses `S2` to remove old elements in the correct FIFO order.

---

7. Why Do We Need Two Stacks?

If we used only one stack, the order would be wrong.

Suppose we push these values into one stack:

```
push(6)
push(3)
push(9)
push(5)
```

The stack becomes:

```
Top
 5
 9
 3
 6
Bottom
```

If we pop from this stack, we get:

```
5
```

But in a queue, the first value should leave first.

The first inserted value was:

```
6
```

So popping directly from one stack gives the wrong result.

That is why we use a second stack.

The second stack helps reverse the order.

---

8. Key Trick: Moving From One Stack to Another Reverses Order

This is the most important idea in Phase 10.

When values move from one stack to another, their order reverses.

Suppose `S1` contains:

```
Top of S1 -> 5, 9, 3, 6 <- Bottom of S1
```

Now move everything from `S1` to `S2`.

Step by step:

```
Pop 5 from S1, push 5 into S2
Pop 9 from S1, push 9 into S2
Pop 3 from S1, push 3 into S2
Pop 6 from S1, push 6 into S2
```

Now `S2` becomes:

```
Top of S2 -> 6, 3, 9, 5 <- Bottom of S2
```

Now `6` is at the top.

That is exactly what we want because `6` entered first.

So the transfer makes the oldest value easy to remove.

---

9. Enqueue Operation

`enqueue()` means:

```
Insert a new value into the queue.
```

In a normal queue, insertion happens at the rear.

In a queue using two stacks, insertion is done by pushing into `S1`.

Enqueue Rule

```
To enqueue x:
    push x into S1
```

Enqueue Code

```
void enqueue(int x) {
    s1.push(x);
}
```

Simple Meaning

```
New values always go into S1.
```

---

10. Enqueue Dry Run

Start with both stacks empty:

```
S1 = empty
S2 = empty
```

Call:

```
enqueue(6)
```

Now:

```
S1 top -> 6
S2 empty
```

Call:

```
enqueue(3)
```

Now:

```
S1 top -> 3, 6
S2 empty
```

Call:

```
enqueue(9)
```

Now:

```
S1 top -> 9, 3, 6
S2 empty
```

Call:

```
enqueue(5)
```

Now:

```
S1 top -> 5, 9, 3, 6
S2 empty
```

The value `6` entered first, but it is at the bottom of `S1`.

So we cannot pop directly from `S1` for dequeue.

---

11. Dequeue Operation

`dequeue()` means:

```
Remove and return the oldest value from the queue.
```

In a queue using two stacks, deletion is handled using `S2`.

Dequeue Rule

```
If S2 has values:
    pop from S2.
```

```
If S2 is empty:
    move all values from S1 to S2.
    then pop from S2.

If both S1 and S2 are empty:
    the queue is empty.
```

---

12. Why Do We Pop From S2 First?

`S2` contains the older values in the correct order.

If `S2` has values, those values entered the queue before the values currently in `S1`.

So they must leave first.

Important rule:

```
Never transfer from S1 to S2 while S2 still has values.
```

Why?

Because `S2` already contains older values. Those older values must be removed before newer values.

---

13. Dequeue Code

```cpp
int dequeue() {
    if (s2.empty()) {
        if (s1.empty()) {
            cout << "Queue is Empty\n";
            return -1;
        }

        while (!s1.empty()) {
            s2.push(s1.top());
            s1.pop();
        }
    }

    int x = s2.top();
    s2.pop();
```

```
    return x;
}
```

---

14. Dequeue Code Explanation

Step 1: Check if `S2` is empty

```
if (s2.empty())
```

Simple meaning:

```
Can we delete directly from S2?
```

If `S2` is not empty, we can pop from it immediately.

If `S2` is empty, we may need to transfer values from `S1`.

Step 2: If `S2` is empty, check `S1`

```
if (s1.empty()) {
    cout << "Queue is Empty\n";
    return -1;
}
```

Simple meaning:

```
If both S1 and S2 are empty, the queue has no elements.
```

So deletion is impossible.

We return `-1` as a failure signal.

Step 3: Transfer all values from `S1` to `S2`

```
while (!s1.empty()) {
    s2.push(s1.top());
```

```
    s1.pop();
}
```

Simple meaning:

```
Move every value from S1 into S2.
```

This reverses the order.

After the transfer, the oldest value becomes the top value of `S2`.

Step 4: Pop from `S2`

```
int x = s2.top();
s2.pop();
return x;
```

Simple meaning:

```
Take the oldest value from S2 and return it.
```

This is the dequeue result.

---

15. Full Dry Run

Now let us follow the full Phase 10 example.

Operations:

```
enqueue(6)
enqueue(3)
enqueue(9)
enqueue(5)
dequeue()
dequeue()
enqueue(4)
enqueue(2)
dequeue()
dequeue()
```

```
dequeue()
dequeue()
```

---

16. Dry Run Step 1: Start

At the beginning:

```
S1 = empty
S2 = empty
Queue = empty
```

---

17. Dry Run Step 2: Enqueue 6

Operation:

```
enqueue(6)
```

Push `6` into `S1`.

```
S1 top -> 6
S2 empty
```

Logical queue:

```
6
```

---

18. Dry Run Step 3: Enqueue 3

Operation:

```
enqueue(3)
```

Push `3` into `S1`.

```
S1 top -> 3, 6
S2 empty
```

Logical queue:

```
6, 3
```

---

19. Dry Run Step 4: Enqueue 9

Operation:

```
enqueue(9)
```

Push `9` into `S1`.

```
S1 top -> 9, 3, 6
S2 empty
```

Logical queue:

```
6, 3, 9
```

---

20. Dry Run Step 5: Enqueue 5

Operation:

```
enqueue(5)
```

Push `5` into `S1`.

```
S1 top -> 5, 9, 3, 6
S2 empty
```

Logical queue:

```
6, 3, 9, 5
```

Notice:

```
The logical queue starts with 6.
But 6 is at the bottom of S1.
```

So we need transfer logic for dequeue.

---

21. Dry Run Step 6: First Dequeue

Operation:

```
dequeue()
```

Check `S2`:

```
S2 is empty
```

So transfer all values from `S1` to `S2`.

Before transfer:

```
S1 top -> 5, 9, 3, 6
S2 empty
```

After transfer:

```
S1 empty
S2 top -> 6, 3, 9, 5
```

Now pop from `S2`.

Deleted value:

```
6
```

After deletion:

```
S1 empty
S2 top -> 3, 9, 5
```

Output:

```
6
```

This is correct because `6` entered first.

---

22. Dry Run Step 7: Second Dequeue

Operation:

```
dequeue()
```

Check `S2`:

```
S2 has values
```

So we do not transfer.

Current state:

```
S1 empty
S2 top -> 3, 9, 5
```

Pop from `S2`.

Deleted value:

```
3
```

After deletion:

```
S1 empty
S2 top -> 9, 5
```

Output so far:

```
6
3
```

---

23. Dry Run Step 8: Enqueue 4

Operation:

```
enqueue(4)
```

Push `4` into `S1`.

Current state:

```
S1 top -> 4
S2 top -> 9, 5
```

Logical queue:

```
9, 5, 4
```

Important point:

```
9 and 5 are older than 4.
```

So `9` and `5` must still leave before `4`.

---

24. Dry Run Step 9: Enqueue 2

Operation:

```
enqueue(2)
```

Push `2` into `S1`.

Current state:

```
S1 top -> 2, 4
S2 top -> 9, 5
```

Logical queue:

```
9, 5, 4, 2
```

Important point:

```
S2 still has older values, so dequeue must use S2 first.
```

---

25. Dry Run Step 10: Third Dequeue

Operation:

```
dequeue()
```

`S2` is not empty.

Current state:

```
S1 top -> 2, 4
S2 top -> 9, 5
```

Pop from `S2`.

Deleted value:

```
9
```

After deletion:

```
S1 top -> 2, 4
S2 top -> 5
```

Output so far:

```
6
3
9
```

---

26. Dry Run Step 11: Fourth Dequeue

Operation:

```
dequeue()
```

`S2` is still not empty.

Current state:

```
S1 top -> 2, 4
S2 top -> 5
```

Pop from `S2`.

Deleted value:

```
5
```

After deletion:

```
S1 top -> 2, 4
S2 empty
```

Output so far:

```
6
3
```

```
9
5
```

---

27. Dry Run Step 12: Fifth Dequeue

Operation:

```
dequeue()
```

Now `S2` is empty.

But `S1` has values:

```
S1 top -> 2, 4
```

So transfer from `S1` to `S2`.

Before transfer:

```
S1 top -> 2, 4
S2 empty
```

After transfer:

```
S1 empty
S2 top -> 4, 2
```

Now pop from `S2`.

Deleted value:

```
4
```

After deletion:

```
S1 empty
S2 top -> 2
```

Output so far:

```
6
3
9
5
4
```

---

28. Dry Run Step 13: Sixth Dequeue

Operation:

```
dequeue()
```

`S2` has values.

Current state:

```
S1 empty
S2 top -> 2
```

Pop from `S2`.

Deleted value:

```
2
```

After deletion:

```
S1 empty
S2 empty
```

Output:

```
6
3
9
5
```

```
4
2
```

This matches the insertion order.

So the queue using two stacks correctly follows FIFO.

---

29. Full Dry Run Table

Step	Operation	S1 State	S2 State	Output
1	Start	empty	empty	—
2	`enqueue(6)`	`6`	empty	—
3	`enqueue(3)`	`3, 6`	empty	—
4	`enqueue(9)`	`9, 3, 6`	empty	—
5	`enqueue(5)`	`5, 9, 3, 6`	empty	—
6	`dequeue()`	empty	`3, 9, 5`	`6`
7	`dequeue()`	empty	`9, 5`	`3`
8	`enqueue(4)`	`4`	`9, 5`	—
9	`enqueue(2)`	`2, 4`	`9, 5`	—
10	`dequeue()`	`2, 4`	`5`	`9`
11	`dequeue()`	`2, 4`	empty	`5`
12	`dequeue()`	empty	`2`	`4`
13	`dequeue()`	empty	empty	`2`

Final output:

```
6 3 9 5 4 2
```

---

30. Complete C++ Program

```cpp
#include <iostream>
#include <stack>
using namespace std;
```

```
class QueueUsingStacks {
private:
    stack<int> s1; // enqueue stack
    stack<int> s2; // dequeue stack

public:
    void enqueue(int x) {
        s1.push(x);
    }

    int dequeue() {
        if (s2.empty()) {
            if (s1.empty()) {
                cout << "Queue is Empty\n";
                return -1;
            }

            while (!s1.empty()) {
                s2.push(s1.top());
                s1.pop();
            }
        }

        int x = s2.top();
        s2.pop();
        return x;
    }
};

int main() {
    QueueUsingStacks q;

    q.enqueue(6);
    q.enqueue(3);
    q.enqueue(9);
    q.enqueue(5);

    cout << q.dequeue() << endl; // 6
    cout << q.dequeue() << endl; // 3

    q.enqueue(4);
    q.enqueue(2);

    cout << q.dequeue() << endl; // 9
    cout << q.dequeue() << endl; // 5
    cout << q.dequeue() << endl; // 4
    cout << q.dequeue() << endl; // 2
```

```
    return 0;
}
```

---

31. Explanation of the Complete Program

Header Files

```
#include <iostream>
#include <stack>
```

`iostream` is used for input and output, such as `cout`.

`stack` is used because the program uses the built-in C++ stack container.

Class Declaration

```
class QueueUsingStacks {
```

This creates a class named `QueueUsingStacks`.

The class represents a queue that is internally built using two stacks.

Private Stacks

```
private:
    stack<int> s1;
    stack<int> s2;
```

These stacks are private because outside code should not directly change them.

Simple meaning:

```
The user should use enqueue() and dequeue(), not directly access S1 and S2.
```

This protects the queue behavior.

Enqueue Function

```
void enqueue(int x) {
    s1.push(x);
}
```

This function inserts a new value into the queue.

It simply pushes the value into `S1`.

This is fast because pushing into a stack is `O(1)`.

Dequeue Function

```
int dequeue() {
```

This function removes and returns the oldest value.

It uses `S2` when possible.

If `S2` is empty, it transfers values from `S1` to `S2`.

Empty Queue Check

```
if (s2.empty()) {
    if (s1.empty()) {
        cout << "Queue is Empty\n";
        return -1;
    }
}
```

This checks whether both stacks are empty.

If both are empty, the queue has no value to delete.

Transfer Loop

```
while (!s1.empty()) {
    s2.push(s1.top());
```

```
    s1.pop();
}
```

This loop moves every value from `S1` to `S2`.

This reverses the order.

After this transfer, the oldest value becomes the top value of `S2`.

Return Deleted Value

```
int x = s2.top();
s2.pop();
return x;
```

This saves the top value of `S2`, removes it, and returns it.

That value is the oldest queue value.

---

32. Program Output

The program prints:

```
6
3
9
5
4
2
```

This output proves FIFO behavior because the values leave in the same order they were logically inserted into the queue.

---

33. Time Complexity

Operation	Time Complexity	Why?
`enqueue()`	`O(1)`	It only pushes one value into `S1`

Operation	Time Complexity	Why?
`dequeue()` when `S2` has values	`O(1)`	It only pops one value from `S2`
`dequeue()` when `S2` is empty and `S1` has values	`O(n)`	It must transfer all values from `S1` to `S2`
Space complexity	`O(n)`	All queue elements are stored inside the two stacks

---

34. Why Enqueue Is O(1)

`enqueue()` only does this:

```
s1.push(x);
```

It does not use a loop. It does not move many elements. It only inserts one value.

So:

```
enqueue() = O(1)
```

---

35. Why Dequeue Can Be O(1)

If `S2` already has values, dequeue only does this:

```
int x = s2.top();
s2.pop();
return x;
```

It removes one value. No loop is needed.

So in this case:

```
dequeue() = O(1)
```

---

36. Why Dequeue Can Be O(n)

If `S2` is empty, we must move all values from `S1` to `S2`.

Example:

```
S1 has 5 values.
S2 is empty.
```

The dequeue function must transfer all 5 values.

That takes time depending on how many values are in `S1`.

So in this case:

```
dequeue() = O(n)
```

---

37. Amortized O(1) Idea

Even though one dequeue can take `O(n)`, the average performance over many operations is still good.

Why?

Because each element is transferred from `S1` to `S2` only once.

Example:

```
6 moves from S1 to S2 only once.
3 moves from S1 to S2 only once.
9 moves from S1 to S2 only once.
5 moves from S1 to S2 only once.
```

After an element is transferred, it stays in `S2` until it is removed.

So the expensive transfer does not happen for every dequeue.

This is called:

```
amortized O(1)
```

Simple meaning:

```
One operation may be expensive, but the average cost over many operations is
small.
```

Beginner-friendly idea:

```
A dequeue may sometimes do a lot of work, but not every dequeue does a lot of
work.
```

---

38. Space Complexity

The queue uses two stacks.

If the queue contains `n` values, those values are stored across `S1` and `S2`.

Some values may be in `S1`. Some values may be in `S2`.

But the total number of stored values is still `n`.

So space complexity is:

```
O(n)
```

---

39. Important Rules to Remember

Rule 1: New values always go into `S1`

```
Enqueue means push into S1.
```

Rule 2: Dequeue uses `S2` first

```
If S2 has values, pop from S2.
```

Rule 3: Transfer only when `S2` is empty

```
If S2 is empty, move all values from S1 to S2.
```

Rule 4: If both stacks are empty, the queue is empty

```
If S1 is empty and S2 is empty, dequeue is not possible.
```

Rule 5: The transfer reverses order

```
Moving S1 to S2 reverses the order and puts the oldest item on top.
```

---

40. Common Beginner Mistakes

Mistake 1: Popping Directly From `S1`

Wrong idea:

```
To dequeue, pop from S1.
```

Why this is wrong:

`S1` is a stack, so it removes the newest value first.

Example:

```
enqueue(6)
enqueue(3)
enqueue(9)
enqueue(5)
```

If we pop from `S1`, we get:

```
5
```

But a queue should return:

```
6
```

So popping directly from `S1` breaks FIFO.

Mistake 2: Transferring Every Time

Wrong idea:

```
Before every dequeue, transfer S1 to S2.
```

Why this is wrong:

If `S2` still has values, those values are older. They must leave first.

Correct rule:

```
Transfer only when S2 is empty.
```

Mistake 3: Forgetting the Empty Case

Before deleting, always check whether the queue is empty.

The queue is empty only when:

```
S1 is empty AND S2 is empty
```

If both are empty, dequeue should not try to pop.

Mistake 4: Thinking Two Stacks Change the Queue Rule

The internal storage uses stacks.

But the outside behavior is still a queue.

So the structure must still follow:

```
FIFO
```

The user of the queue should not care that two stacks are used internally.

Mistake 5: Forgetting That `S2` Contains Older Values

If `S2` contains values, those values came before the values in `S1`.

So `S2` must be emptied before transferring from `S1` again.

---

41. Queue Using Two Stacks vs Earlier Queue Implementations

Implementation	Storage Method	Main Idea
Linear array queue	Array	Uses `front` and `rear` indexes
Circular queue	Circular array	Uses modulo to reuse spaces
Linked-list queue	Nodes	Grows dynamically using heap memory
Deque	Array or linked list	Allows insertion/deletion at both ends
Priority queue	Array/queues/heap idea	Deletes based on priority
Queue using two stacks	Two stacks	Uses stack reversal to create FIFO

The behavior remains queue-like as long as deletion follows FIFO.

---

42. Why Phase 10 Is Important

Phase 10 teaches a deeper data structure idea:

```
One data structure can be used to build another data structure.
```

Here:

```
Two stacks are used to build a queue.
```

This shows that data structures are not just fixed memory layouts. They are also rules and behaviors.

A queue does not have to be stored only as an array or linked list. It can be implemented in different ways.

The important thing is that it behaves correctly.

---

43. ADT Lesson From Phase 10

ADT means:

```
Abstract Data Type
```

An ADT focuses on what a structure does, not how it is built internally.

For a queue ADT, the important behavior is:

```
Insert at rear.
Delete from front.
Follow FIFO.
```

In Phase 10, the internal implementation uses two stacks.

But from the user's point of view, the operations are still:

```
enqueue()
dequeue()
```

So Phase 10 reinforces this idea:

```
The outside behavior matters more than the inside storage method.
```

---

44. Simple Real-Life Analogy

Imagine you have two piles of papers.

Pile 1 is where new papers are placed. Pile 2 is where papers are taken for processing.

New papers go on top of Pile 1.

But if you process directly from Pile 1, you process the newest paper first. That is stack behavior.

To process the oldest paper first, you move all papers from Pile 1 to Pile 2.

This reverses the order.

Now the oldest paper is on top of Pile 2.

That is how two stacks create queue behavior.

---

45. Final Summary

A queue using two stacks works like this:

```
S1 handles enqueue.
S2 handles dequeue.
New values are pushed into S1.
Old values are popped from S2.
If S2 is empty, move all values from S1 to S2.
Moving values from S1 to S2 reverses the order.
The reversal places the oldest value on top of S2.
That makes FIFO possible.
```

The most important final idea is:

```
Two LIFO stacks can work together to create one FIFO queue.
```

---

46. One-Line Memory Trick

Remember this sentence:

```
Push new items into S1, pop old items from S2.
```

Or even shorter:

```
S1 receives, S2 releases.
```

---

47. Final Takeaway

Phase 10 completes the queue journey.

You have now learned that queues can be implemented using many internal structures.

The final lesson is:

```
A data structure is defined by its behavior.
```

So even though stacks are LIFO, two stacks can be arranged to create FIFO behavior.

That is the power of ADT thinking.